UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

GROUP PROJECT PROBLEM DEFINITIONS

SCSM2223 — CROSS-PLATFORM APPLICATION DEVELOPMENT

SEMESTER II, ACADEMIC SESSION 2025/2026

FIELD	DETAILS
Programme	BACHELOR IN SOFTWARE ENGINEERING WITH HONOURS
Group size	FOUR (4) students per team
# Projects	SIX (6) problem definitions — choose ONE per team
Duration	7 weeks of active development
Scope	Full course pipeline — Chapter 1 through Chapter 13
Stack	Vue 3 + Router + Pinia | PHP Slim 4 + PDO + MySQL + JWT | Capacitor

Course Learning Outcomes Addressed

Each project must satisfy CLO1 (apply concepts and components of cross-platform web application development — client-server architecture, frontend frameworks, backend APIs, database integration, deployment), CLO2 (develop an innovative solution for a given problem using appropriate cross-platform technologies), and CLO3 (demonstrate entrepreneurial mindset — resourcefulness in selecting technologies, integrating third-party services, and deploying with mobile adaptability).

Common Requirements (apply to ALL six projects)

Regardless of which problem your team selects, the following requirements MUST be met. They are derived directly from Chapters 1–13 of the syllabus and exercise every CLO.

A. Functional Requirements

- User registration and login with hashed passwords (e.g., bcrypt/Argon2 in PHP).
- JSON Web Token (JWT) authentication for all protected REST endpoints.
- Role-based access (at least TWO roles: end-user and an administrator/owner role).
- RESTful API designed using PHP Slim 4, returning JSON with standard HTTP status codes.
- Persistent storage in MySQL via PHP Data Objects (PDO) using prepared statements only.
- At minimum FOUR (4) related entities (tables) with foreign-key relationships.
- Full Create, Read, Update, Delete (CRUD) for at least TWO (2) primary entities.
- Vue 3 single-page application using Vue Router and a state-management solution (Pinia or Vuex).
- Single File Components, reactive data binding, v-for, v-model, props/emit, lifecycle hooks.
- Responsive UI (mobile, tablet, desktop) using a mobile-first CSS approach.
- Mobile build wrapped with Capacitor for Android (iOS optional).
- Cloud or free-tier deployment of the API + database + frontend, with a public URL.
- Input validation on both client and server sides.
- Defenses against SQL Injection (prepared statements) and XSS (output escaping).
- Error handling that returns helpful JSON error messages and meaningful HTTP codes.

B. Non-Functional Requirements

- Source code under Git/GitHub with a clean commit history showing each member's contribution.
- Project documentation (README, ER diagram, API contract, deployment guide).
- Use of VSCode/ Sublime Text/Sublime Merge, Laragon, GitHub, and Capacitor (per syllabus).
- Code style consistent across the codebase; folder structure follows Vue and Slim conventions.
- Reasonable accessibility (semantic HTML, alt text on images, keyboard navigability).

C. Deliverables (mapped to syllabus assessments)

- PR1 Project Proposal — END OF WEEK 10 of the project (5%): problem statement, objectives, scope, target users, mock-ups, ER diagram, technology stack, 7-week milestone plan, role split.
- PR2 Interim Build — END OF WEEK 12 of the project (10%): static UI complete and a Vue refactor underway; data fetched via Fetch/Axios from a mock JSON source; one full feature demonstrated end-to-end.
- PR3 Final Demo — END OF WEEK 15 of the project (15%): fully integrated working product (Vue + Slim + MySQL + JWT), deployed at a public URL, with the Android build running on emulator or device.
- PR4 Presentation — END OF WEEK 15 of the project (5%): 15-minute live demo with Q&A; every member must speak; include the entrepreneurial pitch.

Project 1 — P1: CampusEats — Pre-Order & Pickup Platform for Campus F&B

"Skip the lunch-hour queue. Order ahead, collect when ready."

1. Problem Background

During lunch and tea breaks, campus cafés and food courts experience long queues that consume 15–25 minutes of student time and reduce vendor throughput. Many vendors prepare food only on-demand, which compounds the wait. Existing food-delivery apps (FoodPanda, GrabFood) are built for off-campus delivery, not on-campus pickup, and the commission rates are too high for small student-run stalls. CampusEats aims to give students a simple way to pre-order from on-campus vendors with a chosen pickup time, while giving vendors a real-time order dashboard.

2. Target Users

- Student Customer — busy student with 30-minute breaks who wants to order ahead and collect food without queueing.
- Vendor / Stall Operator — campus café owner who needs incoming orders to appear on a dashboard, marks items as ready, and tracks daily sales.
- Platform Administrator — manages vendor onboarding, resolves disputes, and views aggregate analytics.

3. Must-Have Features

- Customer browses vendors and menus by category (rice, noodles, drinks, snacks).
- Customer adds items to a cart and places an order with a chosen 15-minute pickup slot.
- Vendor dashboard receives new orders in real time (polling or simple WebSocket).
- Vendor updates order status: placed → preparing → ready → collected.
- Order history for customer and vendor with filters by date/status.
- JWT-based authentication with separate routes for customer / vendor / admin.
- Admin module to approve vendors, deactivate stalls, view sales summary.
- Responsive web UI and Capacitor Android build for the customer side.

4. Should-Have Features

- Search and filter (price range, vegetarian/halal tags, popularity).
- Star ratings and written reviews per vendor.
- Daily promotions / 'Today's special' carousel.
- Notification (email or in-app) when an order is marked ready.

5. Suggested Data Model

Tables (an arrow → indicates a foreign-key relationship):

- `User(id, name, email, password_hash, role, created_at)`
- `Vendor(id, owner_id→User, name, location, opening_hours, is_active)`
- `MenuItem(id, vendor_id→Vendor, name, description, price, category, image_url, in_stock)`
- `Order(id, user_id→User, vendor_id→Vendor, status, total, pickup_at, created_at)`

- `OrderItem(id, order_id→Order, menu_item_id→MenuItem, qty, unit_price)`
- `Review(id, user_id→User, vendor_id→Vendor, rating, comment, created_at)`

6. Entrepreneurial Angle (CLO3)

Revenue model: small flat per-order fee for vendors, optional 'featured stall' upgrade, and an analytics tier. Reflect on real-world adoption — what would convince an actual UTM canteen to onboard?

7. Stretch / Innovation Goals

- Loyalty points and referral codes.
- Pre-order scheduling (e.g., breakfast pre-set the night before).
- Halal-cert badge & nutrition info.

Project 2 — P2: SkillSwap — Peer-to-Peer Tutoring & Skill Exchange Marketplace

"Learn from a senior. Teach a junior. Earn while you study."

1. Problem Background

Students who excel in Calculus, OOP, Vue.js, or even non-academic skills (Photoshop, Cantonese, public speaking) can earn supplementary income by tutoring peers, but there is no purpose-built platform for student-to-student tutoring. Existing freelance platforms (Upwork, MyEduSolve) are aimed at professionals or paid courses and don't suit the per-hour, on-campus, peer-tutoring use case. SkillSwap connects learners and tutors within and across faculties.

2. Target Users

- Learner — struggling with a course or wanting to pick up a non-academic skill, looking for an affordable hourly tutor.
- Tutor — student with strong knowledge in a subject who wants to monetise their skill and build a portfolio.
- Administrator — vets profiles, removes abusive content, mediates disputes.

3. Must-Have Features

- User profile with photo, bio, faculty/year, and a list of skills offered (for tutors).
- Tutor sets hourly rate, availability slots, and short skill descriptions.
- Learner searches and filters tutors by skill, faculty, price range, and rating.
- Booking workflow: request → tutor accepts/declines → confirmed → completed.
- Review and rating after each completed session.
- JWT auth with role-based access (learner, tutor, admin); a single account can hold both learner and tutor roles.
- Wallet / mock payment ledger reflecting bookings and tutor earnings.
- Responsive UI + Capacitor mobile build.

4. Should-Have Features

- In-app messaging between learner and tutor before booking.
- Calendar view of confirmed sessions with .ics export.
- Skill taxonomy with tags and trending skills section.

5. Suggested Data Model

Tables (an arrow → indicates a foreign-key relationship):

- `User(id, name, email, password_hash, faculty, photo_url, bio)`
- `Skill(id, name, category)`
- `UserSkill(id, user_id→User, skill_id→Skill, hourly_rate, level)`
- `Booking(id, learner_id→User, tutor_id→User, skill_id→Skill, scheduled_at, duration, status, total)`
- `Review(id, booking_id→Booking, rating, comment, created_at)`
- `Message(id, sender_id→User, receiver_id→User, body, sent_at)`

6. Entrepreneurial Angle (CLO3)

Revenue model: 10% platform commission on confirmed sessions, premium tutor profiles, optional verification badge for an annual fee. How would you bootstrap supply (tutors) before demand (learners) ?

7. Stretch / Innovation Goals

- Group sessions (1 tutor, multiple learners) at a discounted rate.
- Recommendation engine that suggests tutors based on a learner's enrolled courses.
- Recorded session uploads (links only) for repeat reference.

Project 3 — P3: GreenStep — Personal Carbon Footprint & Eco-Lifestyle Tracker

"Small daily steps. Measurable climate impact."

1. Problem Background

Climate change is among the most urgent global challenges, and individuals collectively account for a large share of avoidable carbon emissions through transport, diet, and energy choices. Most awareness apps either oversimplify (vague 'green tips') or overwhelm (full life-cycle assessment). GreenStep targets the gap: a friendly daily logger that estimates a user's carbon footprint, suggests one realistic action per day, and lets friends compete on streaks and reductions.

2. Target Users

- Environmentally-curious individual who wants to track and reduce their footprint without becoming an expert.
- Community / group leader (e.g., a college residential block) who organises sustainability challenges.
- Administrator who curates the tip library and verifies challenge results.

3. Must-Have Features

- Daily log: transport (mode + km), meals (veg / mixed / red-meat), electricity (kWh), recycling counts.
- Server-side carbon calculation using configurable per-activity emission factors.
- Dashboard with weekly/monthly charts (Vue + Chart.js).
- Streaks and badges for consecutive low-footprint days.
- Tip library; one randomised tip suggested daily.
- Group/community challenges with collective targets.
- JWT auth, role-based admin for tip and emission-factor management.
- Responsive UI + Capacitor Android build (camera optional for proof).

4. Should-Have Features

- Friend connections and a community leaderboard.
- Photo upload for an 'eco-action' (e.g., reusable cup) to earn bonus points.
- Push reminders (or in-app) to log daily.

5. Suggested Data Model

Tables (an arrow → indicates a foreign-key relationship):

- `User(id, name, email, password_hash, role, joined_at)`
- `ActivityType(id, category, name, unit, kg_co2_per_unit)`
- `ActivityLog(id, user_id→User, activity_type_id→ActivityType, amount, logged_on)`
- `Tip(id, title, body, category, source_url)`
- `Challenge(id, name, description, start_date, end_date, target_co2_reduction)`
- `ChallengeMember(id, challenge_id→Challenge, user_id→User)`

- `Badge(id, name, criteria_json, image_url)`

6. Entrepreneurial Angle (CLO3)

Revenue model: B2B tier for companies running employee sustainability programmes, partnerships with eco-friendly brands for reward fulfilment. How does the team ensure the carbon-factor data stays credible?

7. Stretch / Innovation Goals

- Integration with a public carbon-factor dataset (e.g., UK DEFRA via REST).
- Goal-setting (reduce 20% in 30 days) with progress projection.
- Carbon-offset link-out to a vetted partner.

Project 4 — P4: EventOra — Student Society Event Management & Ticketing Platform

"From idea to QR check-in — run your society events end-to-end."

1. Problem Background

Student societies at UTM and similar universities currently manage events with a patchwork of Google Forms, WhatsApp broadcasts, paper attendance lists, and PDF certificates. Faculty advisors have no consolidated visibility into participation, and students miss events they would have attended because there is no central calendar. EventOra centralises event creation, ticketing, QR check-in, attendance reports, and post-event feedback for the campus.

2. Target Users

- Society Organiser — society committee member who creates and manages events.
- Attendee — student or staff who discovers, registers for, and checks in to events.
- Faculty Administrator — approves society activities and views participation analytics.

3. Must-Have Features

- Society dashboard for event creation, editing, and cancellation.
- Public event listings with categories (academic, sports, cultural, religious) and search.
- Free or paid ticketing with seat limits; mock payment for paid events.
- Each ticket has a unique QR code; mobile build can scan a QR to check attendees in.
- Attendance report with CSV export.
- JWT auth with role-based routes (organiser, attendee, faculty admin).
- Responsive UI + Capacitor Android build (camera permission for QR scan).
- Approval workflow: faculty admin approves new events before they go public.

4. Should-Have Features

- Post-event feedback form with rating and comments, dashboard view of results.
- iCalendar (.ics) export and Google Calendar add-link for attendees.
- Email or in-app notifications for confirmed registrations and event reminders.

5. Suggested Data Model

Tables (an arrow → indicates a foreign-key relationship):

- `User(id, name, email, password_hash, role)`
- `Society(id, name, faculty, advisor_id→User)`
- `Event(id, society_id→Society, title, description, venue, starts_at, ends_at, capacity, price, status)`
- `Ticket(id, event_id→Event, user_id→User, qr_code, status, issued_at)`
- `CheckIn(id, ticket_id→Ticket, checked_in_at, checked_in_by→User)`
- `Feedback(id, event_id→Event, user_id→User, rating, comment)`

6. Entrepreneurial Angle (CLO3)

Revenue model: small ticketing fee for paid events, premium analytics for societies, sponsored event placement for partner brands. What would convince a faculty student-affairs office to mandate the platform?

7. Stretch / Innovation Goals

- Recommendation feed based on past attendance.
- Auto-generated digital certificates after event completion.
- Multi-event 'series' (e.g., a weekly workshop) with single registration.

Project 5 — P5: FixIt — On-Demand Local Home Services Marketplace

"Plumbing, electrical, cleaning — verified locals, one tap away."

1. Problem Background

Students living in off-campus apartments and working young adults frequently need small home-service jobs (plumbing leaks, fan installation, deep cleaning, AC service) but have no trusted way to find a local handyman beyond word-of-mouth or unreliable Facebook groups. FixIt is a two-sided marketplace that connects verified local service providers with customers, with transparent pricing, ratings, and a simple booking workflow.

2. Target Users

- Customer — apartment resident who needs an ad-hoc home service.
- Service Provider — verified handyman/freelancer with one or more service categories.
- Administrator — verifies providers, mediates disputes, monitors safety.

3. Must-Have Features

- Service category catalogue (plumbing, electrical, cleaning, gardening, AC, etc.).
- Provider profile with categories, hourly rate or per-job rate, location, photo, KYC document upload (mock).
- Admin verification workflow before a provider goes live.
- Customer browses and filters providers by category, distance, price, rating.
- Booking workflow: requested → accepted → in-progress → completed → reviewed.
- Job ticket with status updates and final cost confirmation.
- JWT auth with strict role-based access; secure file upload for KYC.
- Responsive UI + Capacitor mobile build.

4. Should-Have Features

- In-app chat between customer and provider after booking is accepted.
- Map view of nearby providers (Leaflet + OpenStreetMap, free tier).
- Tip / rating prompt after job completion.

5. Suggested Data Model

Tables (an arrow → indicates a foreign-key relationship):

- `User(id, name, email, password_hash, role, phone)`
- `ServiceCategory(id, name, description, icon_url)`
- `ProviderProfile(id, user_id→User, bio, location, base_rate, is_verified, kyc_doc_url)`
- `ProviderCategory(id, provider_id→ProviderProfile, category_id→ServiceCategory)`
- `Job(id, customer_id→User, provider_id→ProviderProfile, category_id→ServiceCategory, status, scheduled_at, address, total)`
- `Review(id, job_id→Job, rating, comment, created_at)`
- `Message(id, job_id→Job, sender_id→User, body, sent_at)`

6. Entrepreneurial Angle (CLO3)

Revenue model: 12% commission on completed jobs, optional 'priority listing' for providers, insurance add-on for premium tier. How does the team handle trust and safety when providers visit private homes?

7. Stretch / Innovation Goals

- Provider availability calendar with auto-confirm slots.
- Customer location auto-detect via Capacitor Geolocation API.
- Recurring bookings (e.g., weekly cleaning).

Project 6 — P6: MediMinder — Patient Medication Tracking & Adherence Companion

"Right pill, right time — for patients, caregivers, and clinics."

1. Problem Background

Medication non-adherence — missing doses, taking the wrong one, or stopping early — is a leading cause of preventable hospital readmissions, particularly for elderly patients and those on multi-drug regimens. Existing pillbox apps either lack a caregiver/clinician view or are locked to a specific country's pharmacy ecosystem. MediMinder gives patients a clear daily schedule, sends reminders, and lets a caregiver or attending clinic see adherence at a glance.

2. Target Users

- Patient — manages personal medication schedule, marks doses as taken.
- Caregiver — family member or nurse who monitors one or more patients.
- Clinic Administrator — adds patients, defines prescriptions, generates adherence reports.

3. Must-Have Features

- Patient registration with optional caregiver linkage.
- Prescription entry: drug name, dose, frequency, start/end date, notes.
- Daily schedule view with 'Mark Taken' / 'Skipped' buttons.
- Adherence percentage and 7/30-day chart per medication.
- Caregiver dashboard with all linked patients and their adherence.
- Reminder mechanism (in-app + browser/Capacitor local notifications).
- JWT auth, strict role-based access; never expose other patients' data.
- Audit log of dose status changes (who, when).
- Responsive UI + Capacitor Android build.

4. Should-Have Features

- Refill reminder when stock falls below a threshold.
- Drug-interaction warning using a static lookup table seeded into the DB.
- Printable PDF/CSV adherence summary for clinic visits.

5. Suggested Data Model

Tables (an arrow → indicates a foreign-key relationship):

- `User(id, name, email, password_hash, role, dob)`
- `PatientCaregiver(id, patient_id→User, caregiver_id→User, status)`
- `Medication(id, name, form, strength, default_unit)`
- `Prescription(id, patient_id→User, medication_id→Medication, dose, frequency, start_date, end_date, notes)`
- `DoseLog(id, prescription_id→Prescription, scheduled_at, taken_at, status)`
- `Notification(id, user_id→User, type, body, sent_at, read_at)`

6. Entrepreneurial Angle (CLO3)

Revenue model: B2B sales to clinics or community pharmacies as a paid SaaS tier, with the patient/caregiver app remaining free. How will the team protect highly sensitive health data (encryption, access logs, retention policy) while still meeting the 6-week scope?

7. Stretch / Innovation Goals

- Photo capture on dose log (camera proof for caregiver).
- Multi-language UI (English, Bahasa Melayu, Mandarin).
- Voice-friendly reminders for elderly users.

Appendix A — Suggested Role Distribution (Team of 4)

Every team member MUST contribute to design, code, and testing — these are leads, not silos.

Role	Primary Duties
Member 1 — Frontend Lead	Vue project scaffolding, design system, components, routing, state management, mobile-first CSS, accessibility, screen-by-screen UI build.
Member 2 — Backend & API Lead	PHP Slim project setup, RESTful endpoint design, business logic, JSON request/response shaping, error handling, API documentation.
Member 3 — Database & Security Lead	ER diagram, MySQL schema, PDO data-access layer, prepared statements, JWT issuance/verification, input validation, XSS escaping, session/role policy.
Member 4 — DevOps, Mobile & Integration Lead	Cloud deployment of API + DB + frontend, CI/CD basics, Capacitor Android wrap, environment config, integration testing, demo orchestration.

Appendix B — Deliverable Rubric & CLO Mapping

Deliverable	Weight	Assessment Focus	CLO
PR1 Project Proposal (Week 11)	5%	Problem framing, scope, mock-ups, ER diagram, 7-week milestone plan, role split.	CLO2, CLO3
PR2 Interim Build (Week 13)	10%	Responsive UI complete, Vue refactor in progress, data layer fetching from a mock JSON source; one feature demonstrated end-to-end.	CLO1, CLO2
PR3 Final Demo (Week 15)	15%	Fully integrated Vue + Slim + MySQL + JWT product, deployed publicly, Android build runnable; all must-have features working.	CLO1, CLO2, CLO3
PR4 Presentation (Week 15)	5%	Live demo, articulate problem, solution, technical choices, entrepreneurial pitch.	CLO3

1. HP Slim 4 + PDO + MySQL + JWT on the backend, with Capacitor for the mobile build.
2. Scope MUST start from Chapter 1 — that is, every project goes through the complete cross-platform pipeline (web architecture → HTML/CSS → JavaScript/DOM → AJAX/Fetch → async → Vue → full Vue app → backend → REST API → DB → JWT → security → deploy → mobile), not just the back-end half.

Appendix D — Project Summary Matrix

At-a-glance comparison of the six problems.

Code	Project Name	Problem (excerpt)	Tagline
P1	CampusEats	During lunch and tea breaks, campus cafés and food courts experience long queues that consume 15–25 minutes of student time and reduce vendor throughput. Many vendors prepare food only on-demand, which compounds the w...	Skip the lunch-hour queue. Order ahead, collect when ready.
P2	SkillSwap	Students who excel in Calculus, OOP, Vue.js, or even non-academic skills (Photoshop, Cantonese, public speaking) can earn supplementary income by tutoring peers, but there is no purpose-built platform for student-to-s...	Learn from a senior. Teach a junior. Earn while you study.
P3	GreenStep	Climate change is among the most urgent global challenges, and individuals collectively account for a large share of avoidable carbon emissions through transport, diet, and energy choices. Most awareness apps either o...	Small daily steps. Measurable climate impact.
P4	EventOra	Student societies at UTM and similar universities currently manage events with a patchwork of Google Forms, WhatsApp broadcasts, paper attendance lists, and PDF certificates. Faculty advisors have no consolidated visi...	From idea to QR check-in — run your society events end-to-end.
P5	FixIt	Students living in off-campus apartments and working young adults frequently need small home-service jobs (plumbing leaks, fan installation, deep cleaning, AC service) but have no trusted way to find a local handyman ...	Plumbing, electrical, cleaning — verified locals, one tap away.
P6	MediMinder	Medication non-adherence — missing doses, taking the wrong one, or stopping early — is a leading cause of preventable hospital readmissions, particularly for elderly patients and those on multi-drug regimens. Existing...	Right pill, right time — for patients, caregivers, and clinics.